

COCKROACH CRAWLER / 0.7 RELEASE CANDIDATE

A governed, evidence-preserving web acquisition layer for AI agents

Bound reach. Preserve evidence. Reject the candidate when the frozen gate fails.

Author: Ajnas N B

Manuscript: v0.7.0-rc.1

Date: 8 August 2026

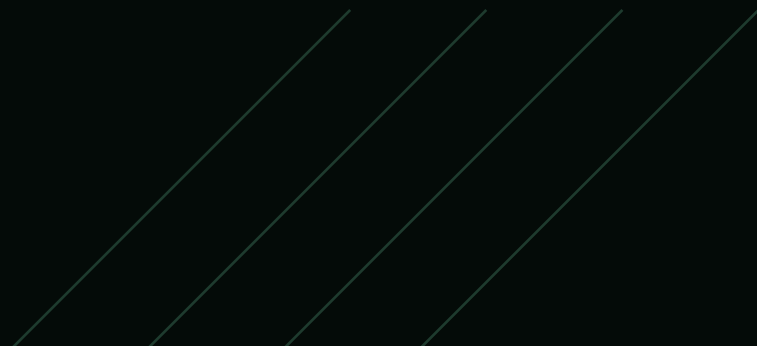
Candidate commit: 90825063d447f07345388d040b1428a311109c2b

Published npm baseline: 0.6.1

Software: MIT

Paper: Creative Commons Attribution 4.0 International

Status: Implementation-backed release-candidate white paper. No independent peer review, independent security certification, reserved DOI, or 0.7 release.



Contents

Architecture, evidence protocol, frozen rejection, reproducibility, and archival boundary.

Abstract	3
1. Research question and contribution	3
2. Candidate identity and publication boundary	3
3. System model	4
4. Local Node acquisition boundary	4
4.1 Admission and address handling	4
4.2 Robots and sensitive paths	5
4.3 Resource ceilings and cancellation	5
5. Browser, provider, and serverless boundaries	5
5.1 Browser rendering	5
5.2 Source registry and ordered routing	5
5.3 Restricted serverless profile	6
6. Extraction and evidence records	6
6.1 Dependency-light core	6
6.2 Optional quality subpath	6
6.3 Evidence as untrusted data	6
7. Evaluation design	6
8. Frozen raw-DOM attempt 003	7
9. Threats to validity	7
9.1 Corpus validity	7
9.2 Metric validity	8
9.3 System validity	8
9.4 Publication validity	8
10. Reproducibility protocol	8
11. Archival and citation protocol	9
12. Conclusion	9
References	9

Abstract

AI agents need current web evidence, but a generic fetch function merges several different kinds of authority: which origins may be contacted, how redirects are followed, whether browser code can run, which credentials may be used, how much work a request may consume, and what evidence survives after extraction. Cockroach Crawler separates those concerns into explicit Node.js entry points for bounded crawling, source-provider routing, browser-host integration, main-content extraction, agent and MCP interfaces, and a narrower serverless profile. The design goal is not unrestricted reach. It is inspectable acquisition under creator-owned policy, with source URLs, hashes, warnings, failures, statistics, and retrieval provenance retained for downstream reasoning.

This paper documents the architecture at source commit [9082506](#), states the security and deployment boundaries, and defines an evidence gate for promotion of the 0.7 candidate. It also reports a negative frozen experiment. Raw-DOM attempt 003 produced precision 0.860252, recall 0.884690, macro F1 0.844419, required-snippet recall 0.758829, and unwanted-snippet inclusion 0.092846. It violated five declared gates and improved precision in six of ten page types where eight were required. The candidate was therefore rejected. The result is retained as development evidence and supports no integration, release, ranking, best-crawler statement, or universal 0.90 claim. The current published npm baseline remains version 0.6.1.

Keywords: governed web crawling; AI agents; evidence provenance; network policy; SSRF reduction; main-content extraction; reproducible evaluation; negative results

1. Research question and contribution

The research question is: *How can an AI-oriented crawler expose useful web acquisition while keeping network, credential, browser, and resource authority visible and testable?*

Cockroach Crawler answers with a composition of bounded interfaces rather than one ambient crawler object. The candidate makes five contributions:

- A local Node transport that validates admitted URLs, checks complete DNS answers, pins requests to a validated address, follows redirects manually, applies robots policy, and enforces resource ceilings.
- A strict agent-facing schema in which model input can narrow but cannot widen creator-owned origins, browser authority, or numeric budgets.
- Separate provider and source-router contracts that report capability before dispatch and do not convert missing credentials into cookie extraction or undeclared scraping fallback.
- An opt-in Node-only quality extractor whose exact native backend, input bounds, diagnostic state, and fail-closed abstention are explicit. Backend unavailability is an error rather than a silent algorithm change.
- A publication protocol that keeps implementation facts, observed development measurements, frozen confirmatory attempts, and release claims in separate ledgers.

These contributions are implementation claims about the cited source candidate. They are not claims of complete Internet coverage, formal security proof, hosted-service capacity, or superiority over all other crawlers.

2. Candidate identity and publication boundary

Three identities must not be conflated.

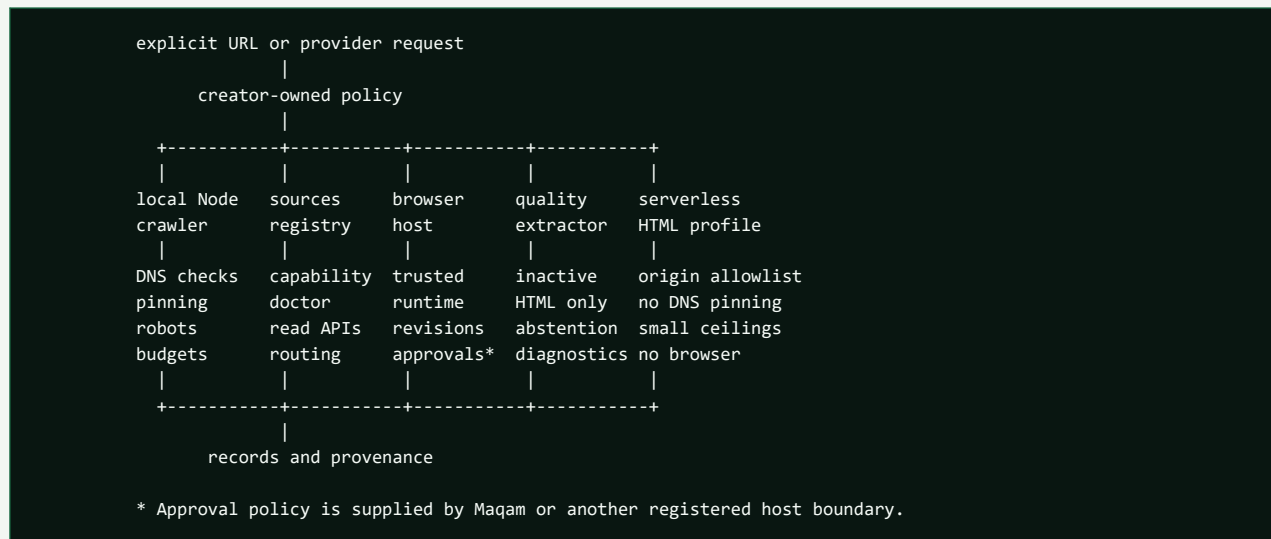
1. The public npm `latest` baseline is `cockroach-crawler@0.6.1` as observed on 8 August 2026.
2. The source tree described here declares package version 0.7.0 and is pinned to commit `90825063d447f07345388d040b1428a311109c2b`.
3. There is no `v0.7.0` tag and no published `cockroach-crawler@0.7.0` package at manuscript freeze time.

The paper therefore uses the term *0.7 release candidate*. A source version string is not a release. Promotion requires agreement among the reviewed source commit, frozen evaluation, release checks, package archive, signed provenance, tag, website, citation metadata, PDF hashes, and authorized repository deposit.

The candidate contains historical development artifacts. Their filenames include `0.7.0` because they were generated for this development line. That filename does not make them release evidence. The evidence status embedded in each artifact and the declared publication gate control the claim.

3. System model

The system begins with an explicit URL or provider request and an operator-defined policy. Five execution paths remain distinct:



The separation is security-relevant. Importing the serverless profile does not load a native quality backend. Importing the quality subpath does not grant network authority. Provider credentials remain inside provider closures and are not copied into normalized records. A browser-host contract does not make browser JavaScript safe, and a crawler result remains untrusted data rather than an instruction channel.

4. Local Node acquisition boundary

4.1 Admission and address handling

The default transport accepts absolute, credential-free HTTP(S) URLs. It rejects non-HTTP schemes, embedded credentials, oversized URLs, loopback, private, link-local, multicast, unspecified, reserved, benchmarking, carrier-grade NAT, IPv4-mapped private addresses, common metadata names, and named cloud metadata endpoints. The complete DNS answer set is classified before contact. Requests are then dispatched to a validated selected address instead of allowing the HTTP client to resolve the hostname again independently.

Every redirect is handled manually and re-enters the same normalization, origin, sensitive-path, DNS, and address checks. This limits a class of time-of-check/time-of-use and redirect-based server-side request forgery paths. It does not constitute a proof against every network or resolver attack. Deployment egress policy remains defense in depth.

4.2 Robots and sensitive paths

Robots retrieval fails closed on network failure and unexpected status. True absence is restricted to the documented not-found responses. Seed pages, crawled links, redirect targets, robots files, sitemap documents, and sitemap entries share the relevant URL-policy path. The CLI and strict agent adapter do not expose a robots bypass.

Sensitive-path matching is a request-reduction mechanism, not an authorization boundary. A permitted `GET` can still produce an unsafe side effect on a badly designed site. Operators remain responsible for authorization and target selection.

4.3 Resource ceilings and cancellation

The crawler exposes hard-bounded controls for seeds, pages, requests, queue entries, links per page, sitemap documents and URLs, URL length, bytes per response, total decoded bytes, redirects, retries, depth, concurrency, request timeout, and total duration. `AbortSignal` cancellation covers DNS, pacing, network work, callbacks, and browser finalization. JavaScript cannot preempt a callback that performs synchronous CPU work, so callbacks are trusted application code.

The distinction between page and request budgets matters. A page budget limits admitted useful outputs. A request budget also counts supporting work such as robots and sitemap retrieval. Both are required to describe the actual cost of a crawl.

5. Browser, provider, and serverless boundaries

5.1 Browser rendering

Optional Playwright rendering installs a context-wide HTTP route before pages are created. HTTP(S) `GET` and `HEAD` traffic is fetched through the validated transport and fulfilled into Chromium. State-changing methods, service workers, downloads, WebRTC, WebTransport, and other escape paths are constrained or denied according to the documented implementation.

Browser mode is not a process sandbox. Hostile JavaScript can consume CPU or memory and can exploit browser defects. Custom browser executables, persistent storage state, and authenticated targets increase operator responsibility. Untrusted browsing should run with process or container isolation and restricted host egress.

The separate `browser-host` entry point implements a structural driver contract for trusted session and page lifecycle, opaque element identifiers, document revisions, previews, post-approval value resolution, and operation deduplication. Maqam remains responsible for approval semantics, replay rejection, and evidence around actions. Installing both packages does not govern direct browser calls that bypass the registered gateway.

5.2 Source registry and ordered routing

The source registry exposes read-oriented `doctor`, `search`, and `read` operations. Provider status is explicit. Missing credentials do not become an implicit browser-session fallback. Optional session-backed social reads require a separately installed operator-controlled runtime and expose fixed read commands rather than a generic shell.

The source router maps a named capability to an ordered provider list. A provider is skipped when its doctor state says the capability is unavailable. Dispatch changes provider only for creator-declared fallback error codes. Authentication, invalid-response, cancellation, timeout, and response-size failures remain non-fallbackable so a serious failure cannot silently drift into a less governed route.

5.3 Restricted serverless profile

The Fetch-compatible serverless profile requires deployment-configured HTTPS origins, manual redirect checks, robots policy, a bearer secret in the Worker template, a rate-limit binding, and small resource ceilings. It intentionally does not resolve, classify, or pin DNS answers. An allowlisted hostname can resolve internally. The profile is therefore suitable only for operator-owned or independently trusted origins combined with infrastructure egress controls. It must not be presented as an arbitrary-origin public proxy.

6. Extraction and evidence records

6.1 Dependency-light core

The core parses responses into bounded records that can include canonical URLs, redirect history, response metadata, links, readable text, Markdown, hashes, warnings, failures, and crawl statistics. Structured extraction supports deterministic selectors and separately bounded host-model adapters. Host-model output must be validated against the configured schema; a model does not inherit network authority from extraction.

6.2 Optional quality subpath

`cockroach-crawler/quality` accepts inactive HTML and delegates main-content extraction to exact `trafilatura@0.2.0`. It validates its input, option object, output, image, metadata, URL, and language bounds. The backend is synchronous and should run in a bounded worker-thread pool when a service cannot tolerate event-loop blocking.

Named profiles and optional `failClosed` admission are visible in the result. An abstained page returns no admitted body and includes reasons. The subpath does not fetch URLs, inherit crawler policy, or fall back to the core when the native dependency is unavailable. Windows, macOS, and glibc Linux on the documented architectures are supported by the upstream prebuilt matrix; Alpine/musl, 32-bit, and other platforms are not claimed.

6.3 Evidence as untrusted data

Crawled HTML and extracted text can contain prompt injection, false claims, or sensitive material. Downstream agents should keep crawler output in a data or evidence channel, preserve its source and hash, and never reinterpret page text as system or developer instructions. Provenance helps identify an input; it does not make that input true.

7. Evaluation design

The project separates four questions because one number cannot answer all of them:

- Main-content quality: How closely does cached-HTML extraction match human-reviewed reference text for a pinned corpus and scorer?
- Admission: When fail-closed mode returns content, what quality and coverage tradeoff does that decision create?
- Conformance: Which exact public robots and URL-normalization vectors pass through the real implementation path?
- Local regression: Does traversal and extraction regress on a deterministic loopback workload under a recorded environment?

Development measurements are useful for diagnosis but cannot confirm the thresholds selected with their help. The project had previously inspected the WCEB partition named `test`, so those rows are labeled observed development evidence. A new release claim requires a frozen candidate, frozen features, source-pinned inputs, declared aggregate and page-type gates, and exactly one authorized outer evaluation after the protocol is fixed.

The evaluator must retain raw per-page rows, dataset identity, split identity, engine and profile flags, abstention state, source fingerprints, and the candidate commit. Aggregate precision without page-type gates can hide regressions concentrated in one content class. A release gate therefore requires both aggregate outcomes and a declared count of page types whose precision improved.

8. Frozen raw-DOM attempt 003

Attempt 003 was the terminal evaluation of a frozen development candidate. The decision rule was fixed before the terminal score. The relevant outcome record is:

- Precision: 0.860252.
- Recall: 0.884690.
- Macro F1: 0.844419.
- Required-snippet recall: 0.758829.
- Unwanted-snippet inclusion: 0.092846.
- Gate violations: 5.
- Page types with precision improvement: 6 of 10; the gate required at least 8 of 10.

The attempt is **rejected**. The system does not round the precision to 0.90, does not substitute recall or F1 for the precision gate, and does not treat a high oracle ceiling as an achieved result. The rejected candidate is not integrated into the release line. The development record is retained because negative results constrain future search and prevent selective reporting.

No statistical uncertainty interval is asserted here because the terminal handoff supplied aggregate and gate outcomes, not a fully reviewed uncertainty analysis. The final evidence package for any future candidate must include the complete receipt required by the frozen protocol before confidence intervals or fold stability are summarized publicly.

9. Threats to validity

9.1 Corpus validity

Cached HTML does not measure live-network behavior, JavaScript rendering, authenticated sessions, OCR, document parsing, or production content drift. WCEB coverage cannot establish universal extraction quality. Previously observed partitions cannot supply fresh confirmation after iteration.

9.2 Metric validity

Word-level precision, recall, and F1 approximate textual overlap. They do not directly measure factual correctness, citation completeness, semantic usefulness, layout preservation, table fidelity, or prompt-injection resistance. Required and unwanted snippet checks cover only annotated literals.

9.3 System validity

Passing unit, browser, type, Worker, and package checks demonstrates behavior for the tested fixtures and environments. It is not a formal proof, penetration test, or independent security certification. Optional upstream tools and provider APIs retain their own defects, terms, quotas, and platform support.

9.4 Publication validity

The author and implementation maintainer are not independent evaluators. This paper is a technical white paper, not peer-reviewed research. A future archival record must preserve the exact manuscript, source hash, PDF hash, builder hash, candidate identity, and evidence receipt so later readers can distinguish versions.

10. Reproducibility protocol

The source candidate can be inspected without treating it as a release:

```
git clone https://github.com/AjnasNB/cockroach-crawler.git
cd cockroach-crawler
git checkout 90825063d447f07345388d040b1428a311109c2b
npm ci --ignore-scripts
npm run release:check
```

The complete release check covers the repository test suite, external TypeScript consumer, local regression fixture, direct dependency-license audit, Chromium integration, Worker types and dry-run bundle, production dependency audit, and package dry run. A successful command verifies the cited candidate under the local environment; it does not override the rejected frozen quality result.

For a future frozen evaluation, the release owner must preserve:

- Candidate commit and dirty-state record.
- Exact dataset revision, source hash, license, split, and page inventory.
- Frozen feature and policy files with cryptographic hashes.
- Evaluator source hash and complete invocation.
- Runtime, operating system, dependency lock, and native backend identity.
- Raw per-page rows, fold or page-type results, aggregate metrics, and gate decisions.
- A signed or hashed receipt that binds every artifact above to one evaluation.

11. Archival and citation protocol

The manuscript source, deterministic PDF, PDF SHA-256, build receipt, [CITATION.cff](#), [codemeta.json](#), and Zenodo draft metadata are versioned together. The paper uses CC BY 4.0; the software remains MIT licensed. The absence of an ORCID, affiliation, or DOI is not filled with invented metadata.

Zenodo publication is a separate authorized action. A DOI should be reserved before the final PDF is frozen so the DOI can be included consistently in the manuscript, citation metadata, and deposition. Publication must not occur until the final files, hashes, creator metadata, license, related identifiers, and preview have been reviewed. Once a record is published, file changes require a new version rather than silent replacement.

At the present candidate state no DOI is claimed. The repository metadata is a draft and must not be interpreted as an existing archival record.

12. Conclusion

Cockroach Crawler's architectural claim is deliberately narrower than "access the web." It provides bounded acquisition paths whose network, credential, browser, extraction, and deployment authorities remain visible. The candidate also demonstrates a research practice: a frozen score that misses its gate is recorded as a rejection, not repaired by rounding or marketing language.

The 0.7 architecture is available for review at the cited commit. The published npm baseline remains 0.6.1. Release promotion, a numerical headline, and a DOI remain blocked until a later immutable candidate passes the declared frozen protocol and the full publication chain agrees.

References

- Ajnas N B. [Cockroach Crawler architecture](#). Candidate source, 2026.
- Ajnas N B. [Cockroach Crawler security policy](#). Candidate source, 2026.
- Ajnas N B. [Cockroach Crawler benchmark method](#). Candidate source, 2026.
- Koster M, Illyes G, Zeller H, Sassman L. [Robots Exclusion Protocol](#). RFC 9309, 2022.
- Berners-Lee T, Fielding R, Masinter L. [Uniform Resource Identifier: Generic Syntax](#). RFC 3986, 2005.
- Fielding R, Nottingham M, Reschke J. [HTTP Semantics](#). RFC 9110, 2022.
- Murrough-Foley. [Web Content Extraction Benchmark](#). Version 1.0 source pinned by the project evaluator.
- Zenodo. [About records](#) and [GitHub software metadata](#). Accessed 8 August 2026.